

قم بتحديث ملف README.md	Ismailu606	b8dd545	قبل يومين
البرامج النصية	...يبيب دبلجة الفيديو بالذكاء الاصطناعي (EN → ES)		الشهر الماضي
src/ video_translator	...يبيب دبلجة الفيديو بالذكاء الاصطناعي (EN → ES)		قبل ثلاثة أسابيع
اختبار	...يبيب دبلجة الفيديو بالذكاء الاصطناعي (EN → ES)		الشهر الماضي
.env.example	...يبيب دبلجة الفيديو بالذكاء الاصطناعي (EN → ES)		الشهر الماضي
.env.macos.example	...يبيب دبلجة الفيديو بالذكاء الاصطناعي (EN → ES)		الشهر الماضي
gitignore	...يبيب دبلجة الفيديو بالذكاء الاصطناعي (EN → ES)		الشهر الماضي
إصدار بايثون	...يبيب دبلجة الفيديو بالذكاء الاصطناعي (EN → ES)		الشهر الماضي
Dockerfile	...يبيب دبلجة الفيديو بالذكاء الاصطناعي (EN → ES)		الشهر الماضي
ملف README.md	قم بتحديث README.md		قبل يومين
ملف docker-compose.yml	...يبيب دبلجة الفيديو بالذكاء الاصطناعي (EN → ES)		الشهر الماضي
pyproject.toml	...يبيب دبلجة الفيديو بالذكاء الاصطناعي (EN → ES)		الشهر الماضي
المتطلبات.txt	...يبيب دبلجة الفيديو بالذكاء الاصطناعي (EN → ES)		الشهر الماضي
قفل الأشعة فوق البنفسجية	...يبيب دبلجة الفيديو بالذكاء الاصطناعي (EN → ES)		الشهر الماضي

ملف التعليمات

خط أنابيب دبلجة الفيديو بالذكاء الاصطناعي – الإنجليزية ← الإسبانية دبلجة الفيديو باستخدام الذكاء الاصطناعي مفتوح المصدر بنسبة 100%

License MIT python 3.10+

سب آي

، إلى فيديو مدبلج متحدثين متعددين مدته ساعة أو أكثر، يضم فيديو إنجليزيًا نظام إنتاج احترافي يحول بالإسبانية، مع استنساخ صوت كل شخص على حدة، مع مراعاة جنسه ونبرته، وترجمة تراعي السياق (المجال، النبرة، المصطلحات). يعمل النظام بالكامل محليًا على نماذج مفتوحة المصدر: لا يتطلب واجهات برمجة تطبيقات مدفوعة، ولا يفصل أي بيانات عن جهازك.

مسار متكامل:

- ، اليت المباشر / VAD للصوت faster-whisper الصوت الأصلي (Whisper) عبر يقوم بنسخ (الطول).
- (ويقدر جنس كل متحدث. pyannote.audio (تحديد اليوميات باستخدام يكتشف من يتحدث ومتى (، وليس مترجم NMT عام: llama.cpp باستخدام لغة محلية (أولاً أو غيرها يترجم مع السياق يمكنك تحديد المجال والنبرة وقاموس المصطلحات الإلزامية، كما أنه يحترم اتفاق الجنس لكل متحدث.
- استنساخ الصوت بدون عينة مع التحكم (IndexTTS-2.5) يقوم باستنساخ صوت كل متحدث على حدة. ومزامنة كل سطر مع خاتمة الزمنية الحقيقية، وضغط الصوت بشكل تكيفي لكل مقطع (الأصلي في المدة بدلاً من قطع المحتوى
- مع الصوت المدبلج (أو الترجمة، إذا رغب المستخدم بذلك)، عبر يقوم هذا البرنامج بعرض الفيديو النهائي ffmpeg .

حقن التبعية (+) Protocol (فصل المجال/التطبيق/البنية التحتية عبر بنية سداسية تم بناؤه باستخدام واختيارات الوحدة مع بدائل الاختيار، والتكامل المستمر على GitHub Actions، وعزل بيئة Python في حالة وجود تعارضات تبعية غير قابلة للحل بين مجموعتين من الذكاء الاصطناعي (تحديد الهوية مقابل الدبلجة) - وهي مشكلة هندسة أنظمة الذكاء الاصطناعي في العالم الحقيقي، وليست مجرد نص برمجي من ملف واحد.

مجموعة الذكاء الاصطناعي

لماذا	تكنولوجيا	منصة
نظام VAD مدمج وسريع ودقيق لمعالجة الصوت الطويل دون الحاجة إلى تقسيم يدوي	(همس كبير-الإصدار 3 أسرع-همس على CTranslate2)	النسخ (STT)
معيار مفتوح المصدر للكشف عن هوية المتحدث وتوقيت كلامه، وهو أساس استنساخ الصوت لكل متحدث على حدة.	(speaker-pyannote.audio diarization-community-1)	تحديد الموقع الجغرافي (اختياري)
بخلاف نموذج الترجمة الآلية العصبية الكلاسيكي (NLLB/M2M100)، يتيح لك توجيه نموذج اللغة الفعّال وفقاً للتعليمات ، مع الترجمة باستخدام سياق اللغة الطبيعية	برنامج LLM مفتوح المصدر عبر llama-1 أو Ollama-server (Qwen2.5, Llama 3.1, gpt-oss, Mistral...)	ترجمة

عن

دبلجة مقاطع الفيديو الإنجليزية إلى الإسبانية باستخدام الذكاء الاصطناعي مع استنساخ الصوت لكل متحدث، والترجمة الواعية بالسياق، ونماذج مفتوحة المصدر محلية بنسبة 100%.

عصر

الذكاء الاصطناعي

الترجمة التلقائية

معالجة الصوت

الدبلجة

الصينية إلى الإنجليزية

مزامنة متأخرة

غرايو

edge-tts

nlb-200

الترجمة الآلية

التوطين

معالجة اللغة الطبيعية

تحويل الكلام إلى نص

تحويل النص إلى كلام

tts

ترجمة الفيديو

معالجة الفيديو

همس-اس

همس

اقرأني

نشاط

النجوم صفر من

مشاهدون لا يوجد

تقراعات 0

مستودع التقارير

الإصدارات

لم يتم نشر أي بيانات صحفية

الطُرود

لم يتم نشر أي حزم

2 المساهمون

Ismailu606

Root1V Victor Espiritu

اللغات

3.3 % صدفه

96.1 % بايثون

0.6 % ملف Dockerfile

ترجمة	برنامج LLM مفتوح المصدر عبر Ollama أو llama-server (Qwen2.5, Llama 3.1, gpt-oss, Mistral...)	بخلاف نموذج الترجمة الآلية العصبية الكلاسيكي (NLLB/M2M100)، يتيح لك توجيه نموذج اللغة الفعّل وفقاً للتعليمات ، مع الترجمة باستخدام سياق اللغة الطبيعية مراعاة توافق المتحدث/الجنس.
دبلجة (صوت إسباني)	(موصى به) IndexTTS-2.5 أو XTTS v2 (Coqui)	استنساخ الصوت لكل متحدث بدون تسجيل ، مصمم للتحكم الأصلي في المدة مسبق + خصيصاً للمزامنة السمعية والبصرية في الدبلجة
وسائط	ffmpeg	استخراج الصوت، والترجمة، وإعادة دمج المسارات الصوتية، ومزج المسارات الصوتية

كل شيء يعمل محلياً؛ لا يتم استدعاء أي واجهة برمجة تطبيقات خارجية مدفوعة الأجر على الإطلاق.

📌 نبیان

بنية سداسية (بنية نظيفة) مع عكس التبعية:

```
src/video_translator/
├── domain/          # Pure entities and exceptions (no external d
│   ├── models.py
│   └── exceptions.py
├── application/     # Use cases + ports (interfaces/Protocols)
│   ├── interfaces.py
│   └── use_cases/translate_video.py  <- pipeline orchestrator
├── infrastructure/  # Concrete adapters (implement the ports)
│   ├── media/ffmpeg_processor.py
│   ├── transcription/faster_whisper_transcriber.py
│   ├── translation/ollama_translator.py  <- context prompt applied
│   ├── synthesis/coqui_tts_synthesizer.py
│   └── subtitles/srt_writer.py
├── config.py        # Typed configuration (pydantic-settings)
├── container.py     # Composition root (dependency injection)
└── cli.py           # Command-line interface (Typer)
```

، بل / infrastructure/ لا يستورد أي شيء من application/ منطق الأعمال (لماذا هذا الهيكل؟ لأن ، وهذا يسمح بما يلي: Protocol يستورد فقط الواجهات) .

- استبدال محرك LLM أو محرك STT أو محرك TTS دون المساس بخط الأنابيب.
- باستخدام نسخ الاختبار الموجودة في الذاكرة (انظر TranslateVideoUseCase الاختبار tests/unit.ffmpeg/GPU إلى لا حاجة إلى) .
- توسيع نطاق الفريق: يتم تطوير كل محول واختباره بشكل منفصل.

🔗 التعامل مع مقاطع الفيديو الطويلة (أكثر من ساعة واحدة)

- يستخدم VAD (الكشف عن نشاط الصوت) داخلياً، حيث يقوم ببث الصوت دون faster-whisper تقسيمه يدوياً أو تحميله بالكامل في الذاكرة.
- ضمن حدود عدد الأحرف، للحفاظ على اتساق `utils/text_batching.py` ، دفعات تتم الترجمة على للترجمات الحديثة عبر الدفعات لضمان يسجل متجدد المصطلحات والأسلوب في الفيديو. ويتم الاحتفاظ اتساق المصطلحات والأسلوب في جميع أنحاء الفيديو.
- يتم التحقق من كل دفعة بنسبة 1:1 (نفس عدد خطوط الإدخال/الإخراج) ويتم إعادة المحاولة تلقائياً (التراجع الآلي) عند حدوث أخطاء في الشبكة أو عدم محاذاة LLM.

🔗 (mode dubbed -- إنشاء فيديو جديد مع صوت مترجم)

يستبدل هذا الوضع صوت الفيديو بالكامل بصوت إسباني مستنسخ من المتحدث الأصلي، ومتزامن مع الوقت - يمكنك تشغيلها كما هي، بنفس الفيديو والصوت الإسباني. mp4. ليس مجرد ترجمة، بل نسخة جديدة تماماً

كود مفتوح المصدر، Bilibili/IndexTeam، [index-tts/index-tts](#)) المحرك الموصى به: IndexTTS-2.5 وأوزان). إنه أحدث ما توصلت إليه تقنية تحويل النص إلى كلام مفتوحة المصدر لهذه الحالة تحديداً للأسباب التالية:

- يقوم باستنساخ نبرة صوت المتحدث من 6-15 ثانية استنساخ الصوت بدون استخدام لقطات صوتية. فقط من الصوت المرجعي.
- دعم اللغة الإسبانية الأصلية (بالإضافة إلى الإنجليزية والصينية واليابانية والعربية).
- (على عكس XTTS v2، الذي يتم إنشاؤه بوتيرة `duration_factor`) التحكم الصريح في المدة حرة ويعتمد على تمديد/ضغط الصوت لاحقاً باستخدام ffmpeg، يمكن لـ IndexTTS-2.5 إنشاء صوت يستهدف مباشرة المدة المستهدفة للمقطع الأصلي - مزامنة أفضل للشفاة/التوقيت وتشويه أقل للبررة.
- التحكم في المشاعر، وهو أمر مفيد حتى لا يبدو الدبلجة باهتة في المحتوى السريدي.

🔗 تثبيت IndexTTS-2.5

غير موجود على PyPI، يتم حله كاعتمادية مسار محلية تشير إلى المستودع المستنسخ (، يقوم البرنامج النصي بآتمنة كل شيء: `pyproject.toml` في `[tool.uv.sources]`) .

```
./scripts/setup_index_tts2.sh
```

uv sync --extra third_party/index-tts ، وتشغيله `third_party/index-tts` يقوم هذا باستنساخ المستودع إلى ملف القتل) وتنزيل نقاط التحقق. uv (الذي يقوم بتنصيبه وتسجيله في `dubbing-indextts`

uv sync --extra third_party/index-tts يقوم هذا باستنساخ المستودع إلى ملف القفل) وتنزيل نقاط التحقق. uv (الذي يقوم بتبنيته وتسجيله في dubbing-indextts

لتثبيت أو تحديث التبعيات، نستخدم `pip sync --extra {whatever}` من الآن فصاعداً. استخدم دائماً `pip install -e {package}` لجعل هذا البرنامج يتتبع `pip install -e {package}` فقط وليس `pip install {package}` كما لا تستخدم أبداً `index.txt` ما عدا ما يعرفه الملف القفل الخاص به. أي حزمة مثبتة خارج هذا الألية (بما في ذلك تلك التي تختفي في المربع) من هذا البرنامج النصي `pip install -e {package}` تمت تثبيتها باستخدام ملف عادي. هذا البرنامج النصي يتعامل معها `pip` على سبب آخر، لأن `pip` القادمة التي تقوم فيها بتشغيل `pip` على أنها "غريبة" عن المشروع.

الاستخدام

```
# extract a 6-15s clean audio sample of the original speaker
ffmpeg -i original_video.mp4 -ss 00:00:10 -t 12 -vn -ar 24000 -ac 1

video-translator translate \
-i original_video.mp4 \
-o ./output \
--mode dubbed \
--speaker-wav voice_sample.wav \
--context "This is a programming tutorial video, warm but professional" \
-v
```

: الفيديو الأصلي مع مسار صوتي إسباني ثانٍ (يتم الاحتفاظ `output/video.dubbed.mp4` . النتيجة هي لاستبداله بالكامل) تم إنشاؤه بواسطة `--no-keep-original-audio` بالأصل كمسار ثانوي افتراضي، `IndexTTS-2.5`، مع نبذة المتحدث الأصلي، متزامنة مقطعاً تلو الآخر مع توقيت النسخ الأصلي.

كيفية تجنب تداخل الأصوات

يتم توليف كل سطر بهدف استغلال الفجوة الحقيقية المتاحة حتى يبدأ السطر التالي (وليس مدة النسخ الأصلية)، وذلك في ثلاث طبقات تبدأ بالعمل بالترتيب التالي:

1. يحاول محرك تحويل النص إلى كلام نفسه توليد الصوت مباشرة عند تلك المدة (التحكم الأصلي في المدة في IndexTTS-2.5).
2. ضمن نطاق لا يشوه الصوت كثيراً. `ffmpeg` إذا استمر التشغيل لفترة طويلة، يتم تعديل السرعة
3. ، يتم قصه إلى الحد الأقصى بالضبط مع تلاشي قصير (80~ ملي ثانية) - ولكن بعد ذلك لم يكن مناسباً ، وليس بشكل مطلق، لذلك يتم ترك المقطع الذي كان مناسباً بالفعل فقط عندما يكون ذلك ضرورياً بالفعل دون تغيير.

يطلب من مترجم لغة الإشارة أيضاً تفضيل الصياغة الموجزة (بطول مماثل للنص الأصلي)، وذلك تحديداً لتقليل (v - الحاجة إلى تدخل الطبقة الثالثة. إذا لاحظت انقطاعات متكررة في الصوت، فراجع السجلات (- فهي توضح لك بدقة مقدار تجاوز كل مقطع للوقت المتاح له، `audio_mixing.segment_truncated` . --context وهو أمر مفيد لتحديد ما إذا كان ينبغي طلب ترجمات أقصر عبر

🔗 [XTTS v2 \(Coqui TTS\)](#): بديل أسهل في التثبيت:

إذا كنت تفضل تجنب استنساخ المستودعات وتنزيل نقاط التحقق يدوياً، فإن XTTs v2 يدعم تبنيته مباشرة (على الرغم من عدم وجود تحكم أصلي في المدة، فقط تعديل ffmpeg بعد ذلك، لذا `uv / pip` باستخدام) فإن المزامنة أقل دقة إلى حد ما:

```
uv sync --extra dubbing-coqui           # requires Python < 3.12
```

```
TTS_BACKEND=coqui_xtts
```

يتطلب (Coqui) استخدام TTS=0.22.0 --extra dubbing-indextxts : غير متوافق مع pandas<2.0 / numpy<2.0 إضافات أحدث - فيها إصدارات أحدث - استخدم أحدهما فقط، وليس كليهما في نفس البيئة. uv حصرية)

--diarize) متحدثون متعددون: صوت كل شخص وجنسه)

لتحديد فإنه يفضل خطوة إضافية --diarize-- إذا كان الفيديو يحتوي على عدة أشخاص يتحدثون، وهي معيار pyannote.audio (الكشف عن هوية المتحدث وتوقيت حديثه) باستخدام المتحدثين مفتوح المصدر لهذا الغرض. ومن ثم، تبدأ عملية المعالجة:

- يُخصّص كل سطر من سطور النص للمحدث المقابل (عن طريق أقصى تدخل زمني).
- يقوم النظام تلقائياً باستخراج أطول فترة كلام لكل محدث تم اكتشافها (15-6 ثانية) كهيئة مرجعية صوتية.
- يقوم بتقدير جنس كل محدث من خلال درجة الصوت (F0)، ويعبرها إلى LLM كسياق حتى يقوم بتعديل التوافق النحوي للجنس في اللغة الإسبانية (على سبيل المثال "liso" مقابل "lisa").
- (وليس صوتاً غائماً) يستأخذ كل محدث بصوته الخاص ، - mode dubbed - في الفيديو واحدًا الفيديو بأكمله؛ يتم توليف سطور الشخص أنبيرة صوت أ، وسطور ب نبيرة صوت ب.

التثنت: سنة افتراضية معزولة (مهم) ☞

بيئة افتراضية منفصلة. فهو يعمل في لا يُعد هذا المشروع تابعًا للمشروع الرئيسي pyannote.audio يتطلب المشروع الرئيسي pyannote.audio يتم استعادتها عبر عملية فرعية، وذلك لسبب وجيه: تمامًا وعمل indexXTS، pyannote-audio-sdk (يعمل القياس عن بُعد الخاص به protobuf>5.0) يتطلب القياس عن بُعد الخاص به) أيضًا! description، audio-tools (يعمل القياس عن بُعد الخاص به 2.5 protobuf<3.20) خطًا "لم يتم دعم تقاض حقيقي، والإصدارات، ولا يوجد أي توافق معها، فيها، في أن. واحد. أنا واجهت

(، بل يُنشئ بيئة افتراضية جديدة (.venv) هذا البرنامج النصي على بيثون الافتراضية الرئيسية لا يؤثر (خصيصاً لهذا الغرض. يستدعي المشروع هذا البرنامج النصي عبر عملية فرعية /venv-diarization/ (انظر استخدامك --diarize عند استخدامك و infrastructure/diarization/subprocess_diarizer.py و scripts/diarization_worker.py (، لذا لا تستورد بيثون الرئيسية البرنامج النصي pyannote.audio . indextxts مباشرةً ولا تتعارض معه أبدًا .

يتطلب أيضًا رمز Hugging Face مع الشروط المقبولة: pyannote.audio

1. hf.co/settings/tokens أنشئ رمزًا مميّزًا على الرابط .
2. [pyannote/speaker-diarization-community-1](https://pyannote.github.io/speaker-diarization-community-1) اقبل شروط النموذج الموجودة على صفحته: .
3. (ستتلقاه العملية الفرعية تلقائيًا). env . في مشروعك الرئيسي HF_TOKEN قم بتعيينه

🔗 libtorchcodec / libavutil.NN.dylib not loaded: إذا رأيت

pyannote.audio >= 4.0 يستخدم torchcodec على torchcodec والذي يعاني على FFmpeg مع Homebrew الخاص بـ macOS من نظام (عدم توافق معروف وغير محلول مع FFmpeg الخاص بـ Homebrew من macOS نظام (- ولا يُحل هذا الخلل بتثبيت إصدار مختلف من Homebrew. الحل الفعال (الذي torchcodec#570 دون للحصول على مكتبات FFmpeg, conda-forge يوصي به فريق torchcodec نفسه) هو استخدام (لأن ذلك سيؤدي إلى تعطيل استخدام Homebrew في سطر الأوامر DYLD_LIBRARY_PATH تصديرها بقية المشروع - فهو يؤثر على أي ملف تنفيذي يرثه، وليس torchcodec فقط). بدلاً ffmpeg / ffprobe ، فقط داخل عملية التمييز المعزولة من ذلك، يقوم المشروع بإضافة هذا المتغير ، فقط داخل عملية التمييز المعزولة من ذلك، يقوم المشروع بإضافة هذا المتغير DIARIZATION_DYLD_LIBRARY_PATH عبر .env :

```
brew install miniforge
conda init zsh # restart your terminal after tl

conda create -n ffmpeg-libs -c conda-forge "ffmpeg<8" -y # <8: what

conda env list # copy the path shown next to "ffmpeg-libs"
```

(.env :) في ملفك /lib قم بتعيين هذا المسار (+)

```
DIARIZATION_DYLD_LIBRARY_PATH=/opt/homebrew/Caskroom/miniforge/base/
```

تأكد من وجود المكتبة بالفعل هناك قبل إعادة المحاولة:

```
ls "/opt/homebrew/Caskroom/miniforge/base/envs/ffmpeg-libs/lib" | gr
```

يعمل من أي طرفية - لا حاجة لتصدير أي شيء يدويًا في كل جلسة. video-translator بعد ضبط ذلك،

🔗 الاستخدام

```
video-translator translate \
-i interview.mp4 \
--mode dubbed \
--diarize \
--context "This is an interview between two people about artificial intelligence" \
-v
```

- يقوم النظام تلقائيًا باستخراج عينة صوتية لكل متحدث يتم اكتشافه. --speaker-wav لا حاجة للتصدير إذا لم يكن لدى المتحدث صوت نقي كافٍ لإنشاء/احتياطي ولا يزال هذا الخيار متاحًا كخيار --speaker-wav عينة صوتية خاصة به، أو لفرض صوت واحد إذا كنت تفضل تعطيل استنساخ الصوت لكل متحدث.

إذا كنت تعرف مسبقًا عدد المتحدثين، فقم بتضييق نطاق البحث:

```
video-translator translate -i video.mp4 --mode dubbed --diarize --min
```

عند الانتهاء، يقوم برنامج CLI بطباعة جدول يحتوي على المتحدثين الذين تم اكتشافهم، وجنسهم المقدّر، وما إذا تم استخراج عينة صوتية قابلة للاستخدام بنجاح لكل منهم.

هو أسلوب استدلال يعتمد على نبذة الصوت (التردد الأساسي)، وليس تحديدًا ملاحظة حول تقدير الجنس: دقيقًا للخصائص الديموغرافية - يُستخدم فقط كإشارة مساعدة للتوافق النحوي في اللغة الإسبانية، وليس (بحيث --mode soft_subtitles كما أنه مفيد بدون دبلجة (مثلًا مع --diarize لتصنيف الأشخاص. تعكس الترجمة نبذة كل متحدث في الترجمة المصاحبة.

🔗 الترجمة المعتمدة على السياق (تطبق على كل من الترجمة المصاحبة والترجمة الصوتية)

مع ما LLM TranslationContext.prompt موجه نظام (أو) في --context يتم إدخال المعامل يلي:

- (--tone formal|informal|technical) المطلوبة النبذة .
- (--glossary glossary.json - الترجمة) الإلزامي مصطلحات المصدر .
- أسماء المنتجات والمصطلحات التقنية وما إلى ذلك.

مثال:

ملف التعليمات

🔍 ✎ ☰

مثال:

```
video-translator translate \
--input pycon_talk.mp4 \
--context "This is a technical PyCon talk about async Python. \
Use a warm but professional tone, aimed at developers. \
Keep terms like 'async', 'coroutine', 'event loop' in English." \
--glossary glossary.json \
--mode soft_subtitles
```

glossary.json :

```
{
  "event loop": "event loop",
  "coroutine": "corrutina",
  "pull request": "pull request"
}
```

تثبيت

متطلبات

- المستخدم للدبلجة، لا يدعم الإصدار 3.13 / Coqui, TTS : ليس 3.13 بايتون 3.10 أو 3.11 أو 3.12)
يغتنه pyproject.toml حتى الآن؛ سيعمل باقي المشروع بشكل جيد على الإصدار 3.13، لكنه
3.11. إعدادات معينة python-version. لتجنب فشل البناء). يتضمن المستودع <3.13
• PATH وهو قيد التشغيل تم تثبيت ffmpeg
• (انظر القسم أدناه) llama-server أو أولاماً نظام خلفي محلي لبرنامج الماجستير في القانون:
• يُنصح باستخدام وحدة معالجة الرسومات من إنفيديا للفيديوهات الطويلة (وحدة المعالجة المركزية تعمل
ولكنها أبطأ بكثير).

(موصى به) UV خطوات مع

```
git clone https://raw.githubusercontent.com/Ismailu606/ai-video-dubb:
cd video-translator

uv python install 3.11      # if you don't already have it (uv downlo
uv sync                     # uses .python-version -> installs with P

# optional dubbing (requires Python < 3.12, already covered by .pyth
uv sync --extra dubbing

cp .env.example .env
```

/ بيئة افتراضية كلاسيكية pip خطوات مع

```
python3.11 -m venv .venv      # explicitly use 3.11 or 3.12, not 3.13
source .venv/bin/activate

pip install -e ".[dev]"       # base install + dev tools
pip install -e ".[dubbing]"   # optional: enables dubbing with Co

cp .env.example .env          # adjust for your hardware

./scripts/setup_models.sh qwen2.5:14b-instruct # downloads the LLM

video-translator check        # verify ffmpeg / ollama (or llama-:
```

الاستخدام

الترجمة فقط (الأسرع، لا يتطلب معالج رسومات قوي)

```
video-translator translate -i video.mp4 -o ./output \
--context "Nature documentary, slow narrative tone." \
--mode subtitles_only
```

تم دمج الترجمة في الفيديو

```
video-translator translate -i video.mp4 --mode burn_subtitles
```

الترجمة كمسار قابل للتحديد (افتراضي)

```
video-translator translate -i video.mp4 --mode soft_subtitles
```

دبلجة كاملة مع استنساخ الصوت

```
video-translator translate -i video.mp4 --mode dubbed \
--speaker-wav original_speaker_sample.wav \
--context "Corporate video, formal and professional tone."
```

دبلجة كاملة مع استنساخ الصوت

```
video-translator translate -i video.mp4 --mode dubbed \
--speaker-wav original_speaker_sample.wav \
--context "Corporate video, formal and professional tone."
```

Ollama بدلاً من llama-server (llama.cpp) استخدام

، (gpt-oss-20b-mx4 على سبيل المثال llama-server إذا كان لديك نموذج يعمل بالفعل باستخدام
) يتصل بواجهة برمجة LlamaServerTranslator فلن تحتاج إلى Ollama. يوفر المشروع محوّلًا ثانيًا
)، بدلاً من واجهة POST /v1/chat/completions llama-server التطبيقات المتوافقة مع OpenAI
) Ollama (/api/chat). برمجة التطبيقات الأصلية لـ

```
TRANSLATION_BACKEND=llama_server
LLAMA_SERVER_HOST=http://localhost:8080
LLAMA_SERVER_MODEL=gpt-oss-20b-mx4
LLAMA_SERVER_MAX_TOKENS=4096
```

) مع تشغيل الخادم الخاص بك بالفعل llama-server -m gpt-oss-20b-mx4.gguf --port
8080 -c 8192):

```
video-translator check # now validates llama-server instead
video-translator translate -i video.mp4 --mode soft_subtitles
```

فهو يعرف فقط TranslateVideoUseCase لا حاجة لتغيير أي شيء آخر في مسار المعالجة:
، لذا لا يهم ما إذا كان Ollama أو llama-server أو vLLM أو أي نظام خلفي Protocol Translator
يحدد المحول الذي سيتم بناؤه بناءً على container.py آخر متوافق مع OpenAI موجودًا في الأسفل.
TRANSLATION_BACKEND .

llama-server : بعض الأمور التي يجب الانتباه إليها

- (حجم السياق) عند بدء تشغيل الخادم بناءً على حجم دفعة الترجمة الخاصة بك (c -) اضبط
)؛ إذا تجاوزت الدفعة + السجل + مطالبة النظام حجم السياق، TRANSLATION_BATCH_MAX_CHARS
فسيتوقف الخادم أو يفشل.
- انقطاع... LLM response misalignment: expected N lines... إذا لاحظت
الاستجابة الأولية في منتصف الجملة، فهذا يعني أن عملية التوليد قد وصلت إلى
الحد الأقصى (افتراضي 4096) قبل إتمام المعالجة. لذا، ارفع هذه LLAMA_SERVER_MAX_TOKENS
أكبر من c - ، وتأكد من أن قيمة الخادم .env القيمة في ملف الإعدادات
حجم موجه الإدخال. يقوم المترجم أيضًا بإعادة المحاولة تلقائيًا LLAMA_SERVER_MAX_TOKENS
قسم translate_batch_with_recovery بتقسيم الدفعة إلى نصفين عند حدوث ذلك (راجع
)، لذا فإن دفعة واحدة طويلة بشكل غير معتاد لن تعطل خط المعالجة prompting.py
بأكمله. مع ذلك، يُنصح برفع الحد الأقصى إذا تكرر هذا الأمر لتحسين الكفاءة.
- هو نموذج فكتم بتنسيق MXFP4: على جهاز M4 Max بسعة 128 gpt-oss-20b-mx4
جيجابايت، لديك مساحة كبيرة للذاكرة؛ عادةً ما تكون عنق الزجاجة هي سرعة الجيل، وليس الذاكرة.

نظام التشغيل macOS (معالج M1/M2/M3/M4 Apple Silicon)

لا Dockerfile / docker- لا تستخدم لا يوجد معالج رسومات NVIDIA/CUDA على نظام Mac، لذا
(فهي مصممة للعمل مع CUDA، ولا يمكن لـ Docker Desktop على نظام Mac تمرير compose.yml
Metal إلى الحاوية على أي حال). على نظام Mac، يعمل كل شيء بشكل أصلي، باستخدام بيئة افتراضية
(.venv).

```
brew install ffmpeg ollama

python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
pip install -e ".[dubbing]" # optional, dubbing

cp .env.macos.example .env # config already tuned for Apple Si

ollama serve & # if it's not already running as a
./scripts/setup_models.sh qwen2.5:32b-instruct

video-translator check
video-translator translate -i video.mp4 --mode soft_subtitles
```

الاختلافات الرئيسية عن جهاز مزود بمعالج رسومات NVIDIA:

لماذا	على جهاز Mac الخاص بك	عنصر
لا يدعم برنامج CTranslate2 تقنية Metal/MPS، بل يدعم وحدة المعالجة المركزية فقط. على معالجات Apple Silicon، يستخدم	WHISPER_DEVICE=cpu , WHISPER_COMPUTE_TYPE=int8	أسرع همس



ملف التعليمات



لماذا	على جهاز Mac الخاص بك	عنصر
لا يدعم برنامج CTranslate2 تقنية Metal/MPS. بل يدعم وحدة المعالجة المركزية فقط. على معالجات Apple Silicon، يستخدم البرنامج تقنية Accelerate ويعمل بكفاءة حتى int8 على large-v3	WHISPER_DEVICE=cpu , WHISPER_COMPUTE_TYPE=int8	أسرع همس
يدعم نظام أولاما عالجات أبل سيليكون بشكل أصلي.	لا تغييرات في الكود، يتم تسريعها تلقائياً بواسطة Metal	أولاما (ماجستير في القانون)
ج ذاكرة موحدة بسعة 128 جيجابايت، لديك مساحة كافية؛ الطرازات الأكبر تعني جودة ترجمة أفضل.	أو حتى qwen2.5:32b-instruct يمكنك استخدام نماذج أكبر: qwen2.5:72b-instruct / llama3.1:70b-instruct	نموذج LLM
هذا الإصدار الثاني من XTTS على MPS غير مستقر (بسبب عدم تنفيذ بعض العمليات)؛ وحدة لمعالجة المركزية أبطأ ولكنها موثوقة.	TTS_DEVICE=cpu	كوكي تي إس (دبلجة)
إنها مبنية على نظام nvidia/cuda التشغيل Mac؛ كل شيء يعمل بشكل أصلي.	لا تستخدم ملف Dockerfile/compose المرفق	عامل مبناء

مع أجهزةك (M4 Max, 128 جيجابايت)، فإن أفضل خطوة من حيث الجودة مقابل الجهد المبذول هي ترقية طراز LLM بدلاً من القلق بشأن أجهزة Whisper: فعلمية النسخ باستخدام وحدة المعالجة المركزية سريعة بشكل معقول، و Ollama مع Metal ينقل النماذج الكبيرة بسلاسة. int8

NVIDIA لينكس / ويندوز مع وحدة معالجة رسومات Docker

```
docker compose up --build
```

المسار وسياقك في [المسار] data/video.mp4. هذا يُشغل Ollama والتطبيق مغا. ضع الفيديو في [المسار] data/context.txt مع دعم NVIDIA output. سيظهر الناتج في [المسار] output. لا ينطبق على macOS (nvidia-container-toolkit).

الاختبار

```
pytest --cov=video_translator --cov-report=term-missing
ruff check src tests
mypy src
```

تستخدم اختبارات الوحدة نسخاً وهمية (أو نسخاً طبق الأصل) للواجهات الأساسية الأربعة (MediaProcessor, Transcriber, Translator, Subtitlewriter) لذلك يتم تشغيلها في ffmpeg أو وحدة معالجة الرسومات أو LLM حقيقي.

قابلية التوسعة

- (على سبيل المثال باستخدام Translator local : قم بإنشاء فئة جديدة تنفذ ترجمة أخرى LLM container.py بدلاً من Ollama) وقم بتسجيلها في transformers
- عبر الروابط. whisper.cpp (على سبيل المثال Transcriber : تنفيذ محرك STT آخر
- source_lang / target_lang : لا يعتمد خط الأنابيب على زوج اللغات؛ قم بتغيير اللغات الأخرى في TranslationContext

رخصة